

TUGAS Kecil 3 IF2211 STRATEGI ALGORITMA



Disusun Oleh:

Mahatma Brahmana - 13524015

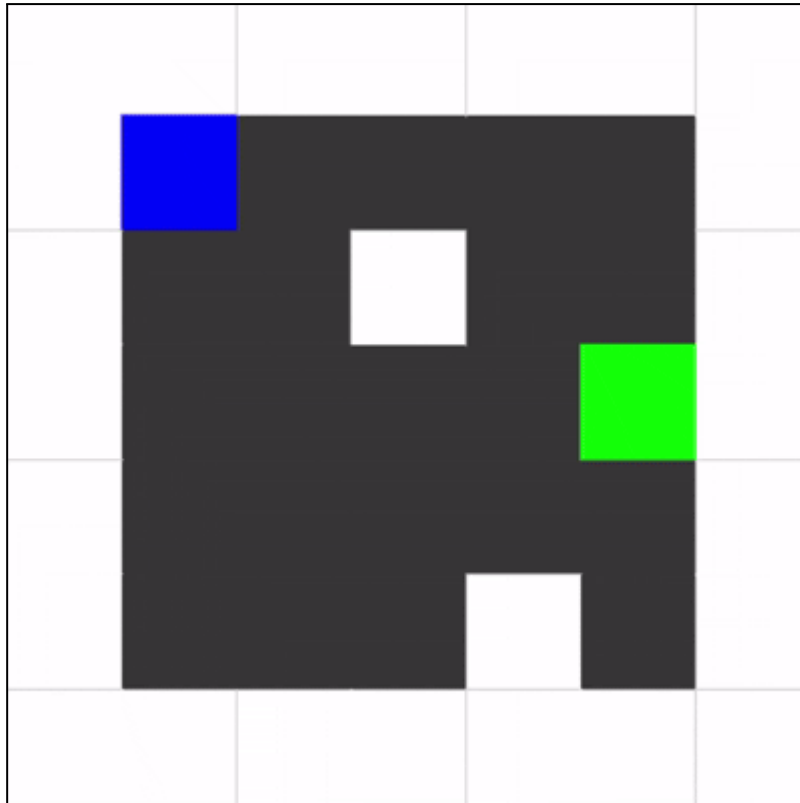
Nathanael Gunawan - 13524066

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025**

Daftar Isi

Daftar Isi.....	2
1. Deskripsi Tugas.....	3
2. Landasan Teori.....	7
2.1. Ice Sliding Puzzle.....	7
2.2. Algoritma Pathfinding.....	7
2.2.1. Algoritma Uniform Cost Search.....	7
2.2.2. Algoritma Greedy Best First Search.....	8
2.2.3. Algoritma A*.....	9
2.2.4. Algoritma BFS.....	9
2.3. Heuristik.....	10
3. Analisis dan Implementasi.....	11
3.1. Pemodelan Masalah dan Aturan Permainan.....	11
3.2. Algoritma Pathfinding.....	17
3.2.1. Algoritma Uniform Cost Search.....	17
3.2.2. Algoritma Greedy Best First Search.....	19
3.2.3. Algoritma A*.....	21
3.2.4. Algoritma BFS.....	24
3.2.5. Perbandingan Teoritis Algoritma.....	26
3.3. Heuristik dan Analisis Admissibility.....	27
3.4. Struktur Program.....	31
3.5. Alur Kerja Program.....	32
4. Hasil Pengujian dan Analisis Hasil.....	33
4.1. Daftar Test Case.....	33
4.2. Hasil Output.....	35
4.3. Statistik Pengujian.....	41
4.4. Analisis Performa Hasil.....	44
4.4.1. Analisis Algoritma.....	44
4.4.2. Analisis Heuristik.....	45
5. Kesimpulan dan Saran.....	46
5.1. Kesimpulan.....	46
5.2. Saran.....	47
6. Daftar Pustaka.....	47
7. Lampiran.....	48
7.1. Pranala Repository Github.....	48
7.2. Pernyataan.....	48
7.3. Checklist.....	48

1. Deskripsi Tugas



Visualisasi Permainan Ice Sliding Puzzle Solver

Ice Sliding Puzzle adalah permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Pin (dalam visualisasi warna biru) hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan licin, karakter tidak akan berhenti bergerak sampai menabrak dinding atau rintangan (dalam visualisasi warna putih).

Spesifikasi Wajib

- Buatlah sebuah program tersebut dalam bahasa Java, Python, C++, C, atau GO yang mengimplementasikan **algoritma Pathfinding (UCS, GBFS, dan A*)** untuk mencari solusi dalam permainan *Ice Sliding Puzzle Solver*
- Tugas dikerjakan **berkelompok** dengan anggota maksimal **2 orang**, boleh lintas kelas maupun kampus. Silahkan isi pendataan kelompok pada link [berikut](#)
- **Input:** program akan memberikan pengguna sebuah arahan pada terminal (jika tidak mengerjakan bonus) untuk memilih file test case berekstensi .txt, kemudian program membaca file test case tersebut yang berisi konfigurasi awal dari *map* Permainan yang akan diselesaikan. Perhatikan bahwa input dapat memiliki ukuran papan yang

berbeda-beda (tidak harus persegi), serta program harus dapat memvalidasi apakah input yang diberikan merupakan input valid atau bukan

Berikut adalah contoh file .txt yang akan dijadikan sebagai input.

Baris pertama memuat 2 angka (N, M) yang menyatakan panjang dan lebar papan

N baris berikutnya merepresentasikan papan

N baris berikutnya merepresentasikan cost untuk melewati tile tersebut

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999
999 6   1   7   2   999 999
999 9   3   4   999 8   999
999 999 999 999 999 999 999
```

Dengan keterangan:

* = path yang bisa dilewati

X = Rintangan/Batu. Aktor akan berhenti tepat sebelum batu.

L = Lava. Melewati lava akan mengalami *game over* (Meskipun tidak berhenti tepat di lava).

Z = Aktor/Pengguna

O = Titik tujuan

<i>= Angka. Ini adalah modifikasi yang ada pada spesifikasi tugas kecil berikut. Aktor harus melewati angka ini sesuai dengan urutan angka di papan. Petak dengan angka 1 harus dilewati setelah aktor melewati petak dengan angka 0, Petak dengan angka 2 harus dilewati setelah aktor melewati petak dengan angka 1, dan seterusnya. (Maksimal hanya sampai angka 9). Apabila aturan ini dilanggar, misalnya petak 1 diinjak sebelum petak 0, maka akan game-over. Setelah petak ini dilewati, maka petak ini akan menjadi petak biasa, dengan kata lain, petak tersebut dapat dilewati lagi tanpa memikirkan constraint ini. Tile ini dianggap sebagai tile normal (bukan batu, artinya, Aktor akan menembus tile ini jika bergerak melaluinya)

Perlu diketahui bahwa, apabila aktor bergerak menuju sisi kanan/atas/kiri/bawah papan, tetapi tidak terdapat rintangan/batu pada sisi tersebut, maka akan mengalami *game over* dan papan ulang dari awal. Pin harus tepat berhenti di titik tujuan dan semua angka telah dilalui sesuai urutan untuk dikatakan sukses.

Setiap tile memiliki cost traversal yang diberikan pada bagian kedua input. Cost ini merepresentasikan biaya yang dikenakan ketika aktor melewati suatu tile. Cost movement dihitung berdasarkan jumlah cost seluruh tile yang dilalui selama satu gerakan sliding, bukan hanya tile tempat aktor berhenti. Tile posisi awal aktor pada awal gerakan tidak dihitung. Namun, tile tujuan akhir gerakan tetap dihitung karena tile tersebut dilalui dan menjadi posisi berhenti aktor.

Contoh:

Jika aktor melakukan gerakan ke kanan dan melewati 3 tile dengan cost 2, 5, dan 4 sebelum berhenti, maka cost gerakan tersebut adalah : $2 + 5 + 4 = 11$.

Total cost solusi adalah penjumlahan seluruh cost movement dari setiap gerakan dalam solusi.

Tile X dan L tetap memiliki cost pada input, tetapi:

- Tile X tidak pernah dilewati karena merupakan rintangan/batu.
- Tile L menyebabkan *game over* jika dilewati, sehingga jalur yang melewati L dianggap tidak valid.

Dengan demikian, cost pada tile X dan L hanya digunakan sebagai bagian dari format input dan tidak dihitung dalam solusi valid

- **Output:**

1. Menampilkan solusi gerakan yang ditemukan
2. Menampilkan cost dari solusi
3. Menampilkan visualisasi dari papan yang di mana oleh Aktor bisa mencapai Titik tujuan dengan algoritma *pathfinding* yang digunakan
4. Menampilkan banyak konfigurasi atau iterasi yang ditinjau oleh algoritma, yang nantinya tiap iterasinya akan dapat disimpan dalam .txt

5. Menampilkan waktu eksekusi program dalam *milisecond* (cukup waktu eksekusi, tidak termasuk membaca dan menulis file).
6. *Playback* (bukan *Live Update*): program harus dapat memvisualisasikan proses *pathfinding* yang dilakukan. Visualisasi proses dilakukan setelah proses pencarian selesai dilakukan. Jika membuat bonus GUI, pastikan playback dapat dijalankan dengan kecepatan yang dapat diatur selayaknya video, serta dapat bergerak maju maupun mundur.

Contoh flow untuk playback pada CLI :

- a. Arrow kiri/kanan untuk maju/mundur step
- b. ESC : User ingin lompat menuju ke step X. Prompt user untuk input suatu angka

Berikut adalah contoh output berdasarkan contoh input diatas

```
>> Masukan file input :
    file/to/input.txt
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
    A*
>> Heuristic apa yang anda pilih? (H1/H2/H3)
    H2
Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87
```

```
Initial
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
```

```
Step 1 : R
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
X**ZX*X
XXXXXXX
```

```
Step 2 : U
XXXXXXX
X0****X
X**X**X
X**Z*OX
X1***LX
X***X*X
XXXXXXX
```

...

```

...
...

Step 8 : R
XXXXXXX
X0****X
X**X**X
X****ZX
X1***LX
X***X*X
XXXXXXX

>> Waktu eksekusi: 10 ms
>> Banyak iterasi yang dilakukan: 9999 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak) :
Ya
>> Pada step berapa anda ingin melakukan playback :
3
>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak) :
Ya
>> Solusi disimpan pada /path/to/solusi.txt

```

2. Landasan Teori

2.1. Ice Sliding Puzzle

Ice Sliding Puzzle adalah permainan puzzle berbasis papan di mana pemain bergerak dengan mekanisme sliding. Ketika pemain memilih suatu arah, pemain akan terus bergerak ke arah tersebut sampai terhalang oleh dinding atau objek tertentu. Karena pemain tidak dapat berhenti bebas di setiap petak, pemilihan arah gerak menjadi bagian penting dalam menentukan solusi.

Permainan ini memiliki posisi awal, posisi tujuan, petak kosong yang dapat dilewati, serta penghalang yang membatasi pergerakan pemain. Pada variasi tertentu, terdapat juga elemen tambahan seperti lava, checkpoint, atau biaya pada setiap petak. Tujuan permainan adalah menemukan urutan gerakan yang membawa pemain dari posisi awal ke posisi tujuan tanpa melanggar aturan permainan.

2.2. Algoritma Pathfinding

2.2.1. Algoritma Uniform Cost Search

Algoritma Uniform Cost Search atau UCS adalah algoritma *pathfinding* yang bertujuan untuk mencari jalur rintangan dengan total biaya paling kecil dari posisi awal sampai posisi akhir. Jadi, UCS

ini memprioritaskan total biaya aktual yang ditempuh sehingga algoritma ini cocok dipakai untuk permasalahan yang tiap perpindahan posisi itu memiliki biaya yang berbeda.

Pada algoritma UCS ini, nilai utama yang dipakai adalah $g(n)$, yang merepresentasikan total biaya aktual dari posisi awal sampai ke posisi tujuan n . Langkah yang diperiksa itu adalah langkah yang mengarah ke posisi yang memiliki nilai $g(n)$ paling kecil. Jadi, fungsi evaluasi UCS ini adalah $f(n) = g(n)$.

Umumnya, UCS menggunakan priority queue sehingga kita bisa mengambil data state dengan nilai $g(n)$ dari yang terkecil terlebih dahulu. Pada konteks di permainan Ice Sliding Puzzle ini, UCS bisa digunakan karena satu gerakan bisa melewati beberapa *tile* sekaligus, yang dimana setiap *tile*-nya itu memiliki cost masing-masing. Dengan ini, UCS ini diharapkan dapat menghasilkan solusi yang optimal berdasarkan total cost lintasan.

2.2.2. Algoritma Greedy Best First Search

Algoritma Greedy Best First Search atau GBFS adalah algoritma *pathfinding* yang memakai informasi heuristik untuk mencari jalan ke posisi tujuan dengan menentukan posisi mana yang tampak paling dekat menuju tujuan akhir. Fokusnya berbeda dengan UCS, GBFS ini tidak berfokus pada total biaya aktual yang sudah ditempuh dari posisi awal. GBFS ini hanya berfokus kepada estimasi biaya dari posisi n saat ini menuju ke posisi tujuan.

Pada algoritma GBFS ini, nilai utama yang digunakan adalah $h(n)$, yang menyatakan perkiraan biaya dari state n menuju state tujuan, yang dimana nilai $h(n)$ ini ditentukan juga berdasarkan heuristik yang digunakan. Fungsi evaluasi GBFS dapat ditulis sebagai $f(n) = h(n)$.

Umumnya, sama dengan UCS, GBFS menggunakan priority queue juga sehingga kita bisa mengambil data state dengan nilai $h(n)$ terkecil terlebih dahulu. Dengan cara ini, pencarian dapat dilakukan dengan lebih cepat apabila heuristik yang digunakan cukup baik. Namun, karena GBFS tidak mempertimbangkan $g(n)$, yaitu biaya aktual sampai ke state n yang sekarang, algoritma ini tidak mengetahui biaya sampai ke jalur ini mahal atau tidak. Hal ini berakibat GBFS tidak dapat selalu menghasilkan solusi dengan total cost paling kecil. Pada konteks di permainan Ice Sliding Puzzle, GBFS

ini dapat selesai dengan cepat karena langsung berusaha mengarah ke checkpoint atau goal, tetapi berisiko memilih jalur yang memiliki total cost yang besar.

2.2.3. Algoritma A*

Algoritma A* adalah algoritma pathfinding yang menggabungkan pertimbangan dari UCS dan GBFS. Algoritma ini bukan hanya melihat total biaya aktual yang sudah ditempuh dari posisi awal, tetapi juga memakai informasi heuristik untuk memperkirakan biaya dari state saat ini menuju state tujuan. Dengan begitu, A* berusaha memilih jalur yang sejauh ini murah, tetapi juga masih terlihat menjanjikan untuk mencapai tujuan.

Pada algoritma A* ini, nilai utama yang digunakan adalah $g(n)$ dan $h(n)$. Nilai $g(n)$ menyatakan total biaya aktual dari posisi awal sampai ke state n , sedangkan $h(n)$ menyatakan perkiraan biaya dari state n menuju menuju state tujuan. Target ini dapat berupa checkpoint atau goal, jika seluruh checkpoint sudah terpenuhi. Kedua nilai tersebut digabungkan dalam fungsi evaluasi, yaitu $f(n) = g(n) + h(n)$.

Umumnya, sama seperti UCS dan GBFS, A* menggunakan priority queue agar program dapat mengambil state dengan nilai $f(n)$ terkecil terlebih dahulu. Dengan langkah ini, algoritma A* dapat melakukan pencarian yang lebih optimal dan terarah dibandingkan UCS karena menggunakan heuristik, juga lebih memperhatikan total cost dibandingkan GBFS karena masih mempertimbangkan $g(n)$. Pada konteks permainan Ice Sliding Puzzle, A* digunakan untuk mencari jalur dengan mempertimbangkan total cost gerakan sliding yang sudah dilakukan serta estimasi jarak menuju checkpoint atau goal. Jika heuristik yang digunakan bersifat admissible, yaitu perkiraan biaya menuju state tujuan yang sesuai, maka A* dapat menghasilkan solusi yang optimal.

2.2.4. Algoritma BFS

Breadth-First Search (BFS) adalah algoritma traversal dan pencarian pada graf yang mengeksplorasi simpul secara melebar. Artinya, BFS akan mengunjungi seluruh simpul yang berada pada kedalaman tertentu terlebih dahulu sebelum melanjutkan ke simpul pada kedalaman berikutnya. Dengan cara ini, BFS membentuk proses pencarian level demi level dari simpul awal.

Dalam konteks pencarian solusi berbasis graf, setiap kemungkinan keadaan dapat direpresentasikan sebagai simpul, sedangkan perpindahan dari satu keadaan ke keadaan lain direpresentasikan sebagai sisi. Pencarian solusi dilakukan dengan menelusuri graf dari simpul awal sampai ditemukan simpul tujuan. Pada permasalahan seperti puzzle, graf yang digunakan sering kali tidak dibentuk secara eksplisit sejak awal, melainkan dibangkitkan secara dinamis selama proses pencarian. Setiap kali suatu simpul diperiksa, algoritma akan membangkitkan simpul-simpul berikutnya yang dapat dicapai dari simpul tersebut.

BFS menggunakan struktur data queue atau antrean yang bekerja dengan prinsip First In First Out (FIFO). Simpul yang pertama kali dimasukkan ke dalam queue akan menjadi simpul pertama yang diproses. Mekanisme ini membuat BFS selalu memproses simpul berdasarkan urutan kedalaman pencarian. Simpul awal dimasukkan terlebih dahulu ke dalam queue, kemudian simpul tersebut dikeluarkan dan diperiksa. Jika simpul tersebut bukan tujuan, seluruh simpul tetangganya dimasukkan ke dalam queue untuk diperiksa kemudian.

2.3. Heuristik

Heuristik adalah fungsi estimasi yang digunakan pada informed search untuk membantu algoritma memilih simpul yang lebih menjanjikan. Berbeda dengan uninformed search yang hanya menggunakan informasi struktur ruang pencarian, informed search memanfaatkan informasi tambahan berupa perkiraan jarak atau biaya dari suatu simpul menuju tujuan.

Dalam algoritma pencarian, heuristik biasanya dinyatakan sebagai $h(n)$, yaitu estimasi biaya dari simpul n ke simpul tujuan. Pada Greedy Best First Search, nilai heuristik digunakan langsung sebagai fungsi evaluasi, $f(n) = h(n)$. Dengan demikian, Greedy Best First Search akan memilih simpul yang terlihat paling dekat dengan tujuan berdasarkan nilai heuristik terkecil.

Pada algoritma A^* , heuristik digabungkan dengan biaya yang sudah ditempuh dari simpul awal. Fungsi evaluasi A^* adalah $f(n) = g(n) + h(n)$ dengan $g(n)$ menyatakan biaya dari simpul awal ke simpul n , $h(n)$ menyatakan estimasi biaya dari simpul n ke tujuan, dan $f(n)$ menyatakan estimasi total biaya lintasan dari simpul awal menuju tujuan melalui simpul n .

Heuristik yang baik dapat mengurangi jumlah simpul yang perlu dieksplorasi karena pencarian menjadi lebih terarah. Namun, kualitas heuristik sangat memengaruhi performa dan optimalitas algoritma. Pada A*, heuristik disebut admissible jika nilai $h(n)$ tidak pernah lebih besar daripada biaya sebenarnya dari n ke tujuan. Dengan kata lain, heuristik admissible selalu memberikan estimasi yang lebih kecil atau sama dengan biaya optimal sebenarnya.

Jika A* menggunakan heuristik admissible dan biaya langkah tidak negatif, maka A* dapat menjamin solusi optimal. Contoh heuristik yang umum digunakan pada permasalahan pathfinding adalah jarak Manhattan, jarak Euclidean, atau straight-line distance, tergantung pada model pergerakan dan aturan masalah yang digunakan.

3. Analisis dan Implementasi

3.1. Pemodelan Masalah dan Aturan Permainan

Masalah pada tugas ini dimodelkan sebagai persoalan pencarian lintasan pada sebuah papan berbentuk matriks berukuran $N \times M$. Setiap elemen pada matriks menggambarkan satu petak permainan. Pemain harus bergerak dari posisi awal menuju sebuah posisi tujuan dengan mengikuti aturan permainan yang telah ditentukan. Selain itu, apabila terdapat checkpoint, pemain harus melewati checkpoint tersebut secara berurutan sebelum mencapai tujuan akhir.

Papan permainan memiliki dua buah matriks, yaitu matriks karakter dan matriks biaya. Matriks karakter menyimpan jenis petak pada papan, sedangkan matriks biaya menyimpan biaya yang diperlukan untuk melewati setiap petak. Dengan pemodelan ini, setiap posisi pada papan dapat dinyatakan sebagai koordinat baris dan kolom, yaitu (row, col). Jenis-jenis petak yang digunakan pada papan adalah sebagai berikut:

1. Z menyatakan posisi awal pemain.
2. O menyatakan posisi tujuan akhir.
3. X menyatakan dinding atau penghalang yang tidak dapat dilewati.
4. L menyatakan lava. Jika pemain melewati petak ini, gerakan dianggap tidak valid.
5. * menyatakan petak kosong yang dapat dilewati.
6. Angka 0 sampai 9 menyatakan checkpoint yang harus dilewati secara berurutan.

Keadaan permainan atau state tidak hanya ditentukan oleh posisi pemain, tetapi juga oleh checkpoint berikutnya yang harus dicapai. Oleh karena itu, sebuah state dimodelkan sebagai tiga komponen, yaitu posisi baris pemain, posisi kolom pemain, dan nomor checkpoint berikutnya. Dalam implementasi, state direpresentasikan sebagai: `State(row, col, next_checkpoint)`

Komponen row dan col menunjukkan posisi pemain pada papan, sedangkan `next_checkpoint` menunjukkan checkpoint berikutnya yang wajib dilewati. Sebagai contoh, jika `next_checkpoint = 2`, maka pemain harus melewati checkpoint bernomor 2 sebelum checkpoint dengan nomor lebih besar dianggap valid.

Gerakan pemain dilakukan dalam empat arah, yaitu atas, bawah, kiri, dan kanan. Namun, pemain tidak hanya bergerak satu petak, melainkan terus meluncur pada arah yang dipilih sampai berhenti karena bertemu dinding. Dengan demikian, satu aksi gerakan dapat menyebabkan pemain melewati beberapa petak sekaligus. Aturan pergerakan pemain adalah sebagai berikut.

1. Pemain dapat memilih salah satu dari empat arah gerakan: atas, bawah, kiri, atau kanan.
2. Setelah arah dipilih, pemain akan terus bergerak pada arah tersebut.
3. Pemain berhenti pada petak terakhir sebelum dinding X.
4. Jika pemain langsung menabrak dinding tanpa berpindah petak, gerakan dianggap tidak valid.
5. Jika pemain keluar dari batas papan, gerakan dianggap tidak valid.
6. Jika pemain melewati lava L, gerakan dianggap tidak valid.
7. Biaya gerakan dihitung dari total biaya seluruh petak yang dilewati selama gerakan tersebut.
8. Jika pemain melewati checkpoint, checkpoint tersebut harus sesuai dengan urutan yang sedang dicari.
9. Checkpoint yang sudah pernah dilewati boleh dilewati kembali.
10. Pemain hanya dianggap berhasil jika berada pada petak tujuan 'O' dan seluruh checkpoint telah dilewati.

Jika pemain melewati checkpoint dengan nomor yang lebih besar dari checkpoint yang sedang dicari, maka gerakan tersebut dianggap tidak valid. Sebaliknya, jika pemain melewati checkpoint yang sesuai, nilai `next_checkpoint` akan dinaikkan sehingga pemain dapat mencari checkpoint berikutnya.

Kondisi awal permainan adalah posisi pemain berada pada petak Z, dengan checkpoint berikutnya bernilai 0. Apabila pada papan tidak terdapat checkpoint, maka pemain dapat langsung mencari jalan menuju tujuan. Kondisi tujuan tercapai apabila posisi pemain sama dengan posisi O dan seluruh checkpoint pada papan telah dilewati.

Dengan pemodelan tersebut, permasalahan dapat dipandang sebagai pencarian pada ruang state. Setiap state merupakan simpul dalam graf implisit, sedangkan setiap gerakan valid dari suatu state menuju state lain merupakan sisi graf dengan bobot sebesar biaya gerakan. Algoritma pencarian seperti BFS, UCS, Greedy Best First Search, dan A* kemudian dapat digunakan untuk mencari solusi berdasarkan aturan dan fungsi evaluasi masing-masing.

Implementasi terdapat dua file, yaitu :

board.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class State:
    row: int
    col: int
    next_checkpoint: int

@dataclass
class BoardInfo:
    rows: int
    cols: int
    board: list[list[str]]
    costs: list[list[int]]
    start: tuple[int, int]
    goal: tuple[int, int]
    checkpoints: dict[int, tuple[int, int]]

def create_board_info(puzzle) -> BoardInfo:
    start = find_start(puzzle)
    goal = find_goal(puzzle)
    checkpoints = find_checkpoint(puzzle)

    return BoardInfo(rows=puzzle.rows,
                      cols=puzzle.cols,
                      board=puzzle.board,
                      costs=puzzle.costs,
                      start=start,
                      goal=goal,
                      checkpoints=checkpoints
    )

def find_start(puzzle) -> tuple[int, int]:
    for r in range(puzzle.rows):
        for c in range(puzzle.cols):
            if puzzle.board[r][c] == "Z":
                return (r,c)

    raise ValueError("Tidak ditemukan Z")

def find_goal(puzzle) -> tuple[int, int]:
    for r in range(puzzle.rows):
        for c in range(puzzle.cols):
            if puzzle.board[r][c] == "0":
                return (r,c)

    raise ValueError("Tidak ditemukan goal")

def find_checkpoint(puzzle) -> dict[int, tuple[int, int]]:
    checkpoints = {}
    for r in range(puzzle.rows):
        for c in range(puzzle.cols):
            tile = puzzle.board[r][c]
            if tile.isdigit():
                checkpoints[int(tile)] = (r, c)

    return checkpoints

```

```

def create_state_awal(board_info: BoardInfo) -> State:
    start_row, start_col = board_info.start
    return State(row=start_row,
                 col=start_col,
                 next_checkpoint=0)

def is_inside(board_info: BoardInfo, row: int, col: int) -> bool:
    return 0 <= row < board_info.rows and 0 <= col < board_info.cols

def is_wall(board_info: BoardInfo, row: int, col: int) -> bool:
    return board_info.board[row][col] == "X"

def is_lava(board_info: BoardInfo, row: int, col: int) -> bool:
    return board_info.board[row][col] == "L"

def get_tile(board_info: BoardInfo, row: int, col: int) -> str:
    return board_info.board[row][col]

def get_cost(board_info: BoardInfo, row: int, col: int) -> int:
    return board_info.costs[row][col]

def get_total_checkpoints(board_info: BoardInfo) -> int:
    return len(board_info.checkpoints)

def is_all_checkpoints_p(board_info: BoardInfo, state: State) -> bool:
    return state.next_checkpoint >= get_total_checkpoints(board_info)

def is_goal(board_info: BoardInfo, state: State) -> bool:
    posisi_pemain = (state.row, state.col)

    return (posisi_pemain == board_info.goal and is_all_checkpoints_p(board_info, state))

def render_board(board_info: BoardInfo, state: State) -> str:
    rendered_rows = []
    for r in range(board_info.rows):
        row_chars = []
        for c in range(board_info.cols):
            tile = board_info.board[r][c]
            if tile == "Z":
                tile = "*"
            if r == state.row and c == state.col:
                tile = "Z"
            row_chars.append(tile)
        rendered_rows.append("".join(row_chars))
    return "\n".join(rendered_rows)

def print_board(board_info: BoardInfo, state: State) -> None:
    print(render_board(board_info, state))

```

movement.py

```

from dataclasses import dataclass
from board import (
    State,
    BoardInfo,
    is_inside,
    is_wall,
    is_lava,
    get_tile,
    get_cost,
)

DIRECTIONS = {
    "U": (-1, 0),
    "D": (1, 0),
    "L": (0, -1),
    "R": (0, 1),
}

@dataclass
class SlideResult:
    valid: bool
    state: State | None
    move_cost: int
    direction: str
    reason: str = ""
    passed_tiles: list[tuple[int, int]] | None = None

def slide(board_info: BoardInfo, state: State, direction: str) -> SlideResult:
    if direction not in DIRECTIONS:
        return SlideResult(valid=False,
                           state=None,
                           move_cost=0,
                           direction=direction,
                           reason=f"Arah gerakan tidak valid: {direction}",
                           passed_tiles=[])

    dr, dc = DIRECTIONS[direction]
    current_row = state.row
    current_col = state.col
    next_checkpoint = state.next_checkpoint
    total_cost = 0
    passed_tiles = []
    while True:
        next_row = current_row + dr
        next_col = current_col + dc
        if not is_inside(board_info, next_row, next_col):
            return SlideResult(valid=False,
                               state=None,
                               move_cost=0,
                               direction=direction,
                               reason="Pemain keluar papan.",
                               passed_tiles=passed_tiles,)

        if is_wall(board_info, next_row, next_col):
            if len(passed_tiles) == 0:
                return SlideResult(valid=False,
                                    state=None, move_cost=0,
                                    direction=direction,
                                    reason="Pemain langsung menabrak dinding dan tidak bergerak.",
                                    passed_tiles=passed_tiles,)

            new_state = State(row=current_row,
                               col=current_col,
                               next_checkpoint=next_checkpoint,)
            return SlideResult(valid=True,
                                state=new_state,
                                move_cost=total_cost,
                                direction=direction,
                                reason="",
                                passed_tiles=passed_tiles,)

        tile = get_tile(board_info, next_row, next_col)

        if is_lava(board_info, next_row, next_col):
            return SlideResult(valid=False,
                               state=None,
                               move_cost=0,
                               direction=direction,
                               reason="Pemain melewati lava.",
                               passed_tiles=passed_tiles + [(next_row, next_col)],)

        total_cost += get_cost(board_info, next_row, next_col)
        passed_tiles.append((next_row, next_col))

        if tile.isdigit():
            checkpoint_number = int(tile)
            if checkpoint_number == next_checkpoint:
                next_checkpoint += 1
            elif checkpoint_number < next_checkpoint:
                pass
            else:
                return SlideResult(valid=False,
                                    state=None,
                                    move_cost=0,
                                    direction=direction, reason=(f"Checkpoint {checkpoint_number} dilewati sebelum
checkpoint {next_checkpoint}."),
                                    passed_tiles=passed_tiles,)

        current_row = next_row
        current_col = next_col

```



```
def get_valid_neighbors(board_info: BoardInfo, state: State) -> list[SlideResult]:
    """
    Menghasilkan semua gerakan valid dari sebuah state.
    Dipakai untuk UCS, GBFS, dan A*.
    """
    neighbors = []
    for direction in DIRECTIONS:
        result = slide(board_info, state, direction)
        if result.valid:
            neighbors.append(result)
    return neighbors
```

3.2. Algoritma Pathfinding

3.2.1. Algoritma Uniform Cost Search

Uniform Cost Search digunakan pada program ini untuk mencari solusi berdasarkan total cost terkecil. Di konteks puzzle Ice Sliding ini, algoritma UCS cocok digunakan karena algoritma ini memilih state berdasarkan biaya aktual yang sudah ditempuh, bukan berdasarkan jumlah gerakan atau perkiraan jarak ke tujuan.

Pada implementasi program, UCS menggunakan priority queue yang diimplementasikan dengan heap melalui modul heapq. Data yang dimasukkan ke dalam heap adalah state beserta total cost aktual dari state awal menuju state tersebut. Heap ini menyimpan kandidat state yang bisa diperiksa, lalu mengurutkannya berdasarkan nilai cost terkecil.

Pada program ini, isi heap pada UCS dapat digambarkan sebagai :

$(cost, heap_id, state)$

Keterangan:

$cost$ = total biaya aktual dari state awal sampai state tersebut

$heap_id$ = nilai tambahan untuk membedakan urutan masuk jika $cost$ sama

$state$ = posisi pemain dan checkpoint berikutnya

State yang diperiksa terlebih dahulu adalah state dengan nilai $cost$ paling kecil. Nilai $cost$ ini sama dengan $g(n)$, yaitu total biaya aktual yang sudah ditempuh dari state awal sampai state n . Jadi, UCS tidak memilih state berdasarkan perkiraan jarak ke tujuan, tetapi berdasarkan biaya asli yang sudah dihitung.

Langkah-langkah UCS yang diimplementasikan adalah sebagai berikut.

1. Membentuk state awal dari posisi Z dengan checkpoint awal yang harus dicapai.
2. Memasukkan state awal ke dalam priority queue dengan cost awal 0.
3. Menyimpan cost terbaik state awal sebagai 0.
4. Mengambil state dengan total cost terkecil dari priority queue.
5. Memeriksa apakah state tersebut sudah memenuhi kondisi goal.
6. Jika belum goal, ambil semua state tetangga yang valid melalui gerakan sliding.
7. Menghitung cost baru untuk setiap tetangga berdasarkan total cost sebelumnya ditambah cost gerakan.
8. Jika tetangga belum pernah digapai atau cost baru lebih kecil dari cost sebelumnya, program memperbarui best_cost dan parent.
9. Memasukkan tetangga tersebut ke dalam priority queue dengan prioritas berdasarkan cost barunya.
10. Mengulangi proses sampai goal ditemukan atau priority queue kosong.
11. Jika goal ditemukan, program membentuk ulang urutan gerakan menggunakan data parent.

Dengan mekanisme ini, UCS selalu memproses state yang memiliki total cost aktual paling kecil terlebih dahulu.

ucs.py

```

import heapq
import time
from board import is_goal
from movement import get_valid_neighbors
from utils import reconstruct_path, make_result, SearchResult

def uniform_cost_search(board_info, start_state) -> SearchResult:
    start_time = time.perf_counter()

    pq = []
    heap_id = 0
    heapq.heappush(pq, (0, heap_id, start_state))

    best_cost = {start_state: 0}
    parent = {start_state: None}
    iterations = 0

    while pq:
        current_cost, _, current_state = heapq.heappop(pq)
        iterations += 1

        if current_cost != best_cost[current_state]:
            continue

        if is_goal(board_info, current_state):
            moves, states = reconstruct_path(parent, current_state)
            time_ms = (time.perf_counter() - start_time) * 1000
            return make_result(True, moves, states, current_cost, iterations, time_ms)

        for neighbor in get_valid_neighbors(board_info, current_state):
            next_state = neighbor.state
            new_cost = current_cost + neighbor.move_cost

            if next_state not in best_cost or new_cost < best_cost[next_state]:
                best_cost[next_state] = new_cost
                parent[next_state] = (current_state, neighbor.direction)
                heap_id += 1
                heapq.heappush(pq, (new_cost, heap_id, next_state))

    time_ms = (time.perf_counter() - start_time) * 1000
    return make_result(False, "", [], None, iterations, time_ms)

```

3.2.2. Algoritma Greedy Best First Search

Greedy Best First Search digunakan pada program ini untuk mencari solusi dengan bantuan heuristic. Pada permasalahan Ice Sliding Puzzle, heuristic dipakai untuk memperkirakan state mana yang terlihat lebih dekat dengan target. Dengan cara ini, GBFS mencoba mengarahkan pencarian agar tidak menyebar terlalu luas seperti pencarian tanpa heuristic.

Pada implementasi program, GBFS menggunakan priority queue yang diimplementasikan dengan heap melalui modul heapq. Data yang dimasukkan ke dalam heap adalah state beserta nilai heuristic dari state tersebut. Berbeda dengan UCS, GBFS tidak menggunakan total cost aktual dari state awal sebagai dasar pemilihan state berikutnya. Heap menyimpan kandidat state yang bisa diperiksa, lalu mengurutkannya berdasarkan nilai heuristic terkecil.

Secara umum, isi queue pada GBFS dapat digambarkan sebagai berikut.

`(heuristic_value, heap_id, state)`

Keterangan:

heuristic_value = estimasi biaya atau jarak dari state menuju target

heap_id = nilai tambahan untuk membedakan urutan masuk jika heuristic sama

state = posisi pemain dan checkpoint berikutnya

State yang diperiksa terlebih dahulu adalah state dengan nilai heuristic_value paling kecil. Nilai ini sama dengan $h(n)$, yaitu estimasi biaya dari state n menuju target. Pada implementasi ini, target heuristic adalah checkpoint berikutnya jika masih ada checkpoint yang belum dilewati, atau goal jika seluruh checkpoint sudah dilewati.

Langkah-langkah GBFS yang diimplementasikan adalah sebagai berikut.

1. Membentuk state awal dari posisi Z.
2. Menghitung nilai heuristic dari state awal.
3. Memasukkan state awal ke dalam priority queue dengan prioritas nilai heuristic.
4. Menyiapkan struktur visited untuk mencatat state yang sudah diproses.
5. Mengambil state dengan nilai heuristic terkecil dari priority queue.
6. Memeriksa apakah state tersebut sudah memenuhi kondisi goal.
7. Jika belum goal, ambil semua state tetangga yang valid melalui gerakan sliding.
8. Menghitung nilai heuristic untuk setiap state tetangga.
9. Jika tetangga belum pernah diproses, program menyimpan data parent.
10. Memasukkan tetangga ke dalam priority queue berdasarkan nilai heuristic-nya.
11. Mengulangi proses sampai goal ditemukan atau priority queue kosong.

12. Jika goal ditemukan, program membentuk ulang urutan gerakan menggunakan data parent.

Dengan mekanisme ini, GBFS cenderung lebih cepat mengarah ke target karena state yang terlihat paling dekat akan diproses lebih dahulu.

gbfs.py

```
import heapq
import time
from board import is_goal
from movement import get_valid_neighbors
from utils import reconstruct_path, make_result, SearchResult

def greedy_best_first_search(board_info, start_state, selected_heuristic) -> SearchResult:
    start_time = time.perf_counter()

    pq = []
    heap_id = 0
    heapq.heappush(pq, (selected_heuristic(board_info, start_state), heap_id, start_state))

    parent = {start_state: None}
    path_cost = {start_state: 0}
    visited = set()
    iterations = 0

    while pq:
        _, _, current_state = heapq.heappop(pq)
        iterations += 1

        if current_state in visited:
            continue
        visited.add(current_state)

        if is_goal(board_info, current_state):
            moves, states = reconstruct_path(parent, current_state)
            time_ms = (time.perf_counter() - start_time) * 1000
            return make_result(True, moves, states, path_cost[current_state], iterations, time_ms)

        for neighbor in get_valid_neighbors(board_info, current_state):
            next_state = neighbor.state

            if next_state in visited or next_state in parent:
                continue

            parent[next_state] = (current_state, neighbor.direction)
            path_cost[next_state] = path_cost[current_state] + neighbor.move_cost

            heap_id += 1
            heapq.heappush(pq, (selected_heuristic(board_info, next_state), heap_id, next_state))

    time_ms = (time.perf_counter() - start_time) * 1000
    return make_result(False, "", [], None, iterations, time_ms)
```

3.2.3. Algoritma A*

A* digunakan pada program ini untuk menggabungkan pertimbangan total cost aktual dan heuristic. Pada Ice Sliding Puzzle, total cost tetap penting karena setiap tile memiliki biaya yang berbeda. Namun, hanya mengandalkan cost aktual seperti UCS dapat membuat pencarian cukup luas. Oleh karena itu, A* menggunakan heuristic agar pencarian lebih terarah, tetapi tetap memperhatikan biaya yang sudah ditempuh.

Pada implementasi program, A* menggunakan priority queue yang diimplementasikan dengan heap melalui modul heapq. Data yang dimasukkan ke dalam heap adalah state beserta nilai prioritas hasil penjumlahan antara cost aktual dan heuristic. Queue menyimpan kandidat state yang bisa diperiksa, lalu mengurutkannya berdasarkan nilai $f(n)$ terkecil.

Secara umum, isi queue pada A* dapat digambarkan sebagai berikut.

$(f_value, heap_id, state)$

Keterangan:

$$f_value = g(n) + h(n)$$

$g(n)$ = total biaya aktual dari state awal sampai state n

$h(n)$ = estimasi biaya atau jarak dari state n menuju target

$heap_id$ = nilai tambahan untuk membedakan urutan masuk jika f_value sama

$state$ = posisi pemain dan checkpoint berikutnya

State yang diperiksa terlebih dahulu adalah state dengan nilai f_value paling kecil. Nilai f_value menunjukkan perkiraan total biaya lintasan jika solusi melewati state tersebut. Dengan cara ini, A* tidak hanya fokus ke state yang terlihat dekat dengan target, tetapi juga mempertimbangkan apakah jalur menuju state tersebut sudah mahal atau belum.

Langkah-langkah A* yang diimplementasikan adalah sebagai berikut.

1. Membentuk state awal dari posisi Z.
2. Memberikan nilai cost awal atau $g(n)$ sebesar 0 pada state awal.
3. Menghitung nilai heuristic atau $h(n)$ dari state awal.
4. Memasukkan state awal ke dalam priority queue dengan prioritas $g(n) + h(n)$.
5. Menyimpan cost terbaik state awal dan data parent.
6. Mengambil state dengan nilai prioritas terkecil dari priority queue.
7. Memeriksa apakah state tersebut sudah memenuhi kondisi goal.

8. Jika belum goal, ambil semua state tetangga yang valid melalui gerakan sliding.
9. Untuk setiap tetangga, menghitung cost baru dari state awal menuju tetangga tersebut.
10. Jika tetangga belum pernah dicapai atau cost baru lebih kecil dari cost sebelumnya, program memperbarui cost terbaik dan data parent.
11. Menghitung nilai heuristic tetangga berdasarkan target yang sesuai.
12. Memasukkan tetangga ke dalam priority queue dengan prioritas cost baru ditambah nilai heuristic.
13. Mengulangi proses sampai goal ditemukan atau priority queue kosong.
14. Jika goal ditemukan, program membentuk ulang urutan gerakan menggunakan data parent.

Dengan mekanisme ini, A* menjadi lebih seimbang dibandingkan UCS dan GBFS.

astar.py

```
import heapq
import time
from board import is_goal
from movement import get_valid_neighbors
from utils import reconstruct_path, make_result, SearchResult

def a_star_search(board_info, start_state, selected_heuristic) -> SearchResult:
    start_time = time.perf_counter()

    pq = []
    heap_id = 0

    best_cost = {start_state: 0}
    parent = {start_state: None}

    start_value = selected_heuristic(board_info, start_state)
    heapq.heappush(pq, (start_value, 0, heap_id, start_state))
    iterations = 0

    while pq:
        _, current_g, _, current_state = heapq.heappop(pq)
        iterations += 1

        if current_g != best_cost[current_state]:
            continue

        if is_goal(board_info, current_state):
            moves, states = reconstruct_path(parent, current_state)
            time_ms = (time.perf_counter() - start_time) * 1000
            return make_result(True, moves, states, best_cost[current_state], iterations, time_ms)

        for neighbor in get_valid_neighbors(board_info, current_state):
            next_state = neighbor.state
            new_cost = best_cost[current_state] + neighbor.move_cost

            if next_state not in best_cost or new_cost < best_cost[next_state]:
                best_cost[next_state] = new_cost
                parent[next_state] = (current_state, neighbor.direction)

                total_value = new_cost + selected_heuristic(board_info, next_state)
                heap_id += 1
                heapq.heappush(pq, (total_value, new_cost, heap_id, next_state))

    time_ms = (time.perf_counter() - start_time) * 1000
    return make_result(False, "", [], None, iterations, time_ms)
```

3.2.4. Algoritma BFS

Breadth-First Search (BFS) memeriksa seluruh kemungkinan gerakan dengan jumlah langkah paling sedikit terlebih dahulu sebelum memeriksa kemungkinan gerakan dengan jumlah langkah yang lebih banyak.

Pada permasalahan Ice Sliding Puzzle, setiap state merepresentasikan posisi pemain pada papan serta checkpoint berikutnya yang harus dilewati. Suatu state dinyatakan sebagai posisi baris, posisi kolom, dan nomor checkpoint berikutnya. Dari satu state, algoritma akan membangkitkan seluruh state tetangga yang dapat dicapai melalui gerakan valid ke atas, bawah, kiri, atau kanan.

BFS menggunakan struktur data queue. State awal dimasukkan ke dalam queue sebagai titik awal pencarian. Selanjutnya, selama queue belum kosong, algoritma mengambil state paling depan untuk diperiksa. Jika state tersebut merupakan state tujuan, yaitu pemain sudah berada di petak tujuan dan seluruh checkpoint telah dilewati, maka pencarian dihentikan dan jalur solusi direkonstruksi.

Jika state yang diperiksa belum merupakan tujuan, algoritma akan membangkitkan semua gerakan valid dari state tersebut. Gerakan valid adalah gerakan yang tidak keluar dari papan, tidak melewati lava, tidak langsung menabrak dinding tanpa berpindah, dan tidak melanggar urutan checkpoint. Setiap state hasil gerakan yang belum pernah dikunjungi akan dimasukkan ke dalam queue untuk diperiksa pada iterasi berikutnya.

Dalam implementasi, BFS juga menyimpan informasi parent dari setiap state. Informasi ini digunakan untuk membentuk kembali jalur solusi setelah state tujuan ditemukan. Selain itu, program juga menyimpan total cost lintasan menuju setiap state. Namun, cost tersebut tidak digunakan sebagai dasar pemilihan state berikutnya pada BFS. BFS tetap memilih state berdasarkan urutan masuk ke dalam queue, bukan berdasarkan biaya lintasan.

Langkah-langkah BFS yang diimplementasikan adalah sebagai berikut.

1. Membuat state awal dari posisi Z dengan checkpoint berikutnya bernilai 0.
2. Memasukkan state awal ke dalam queue.

3. Menandai state awal sebagai state yang sudah dikunjungi.
4. Mengambil state paling depan dari queue.
5. Memeriksa apakah state tersebut merupakan state tujuan.
6. Jika sudah mencapai tujuan, jalur solusi dibentuk ulang menggunakan data parent.
7. Jika belum mencapai tujuan, semua gerakan valid dari state tersebut dibangkitkan.
8. Setiap state baru yang belum pernah dikunjungi dimasukkan ke queue.
9. Proses berulang sampai ditemukan solusi atau queue menjadi kosong.
10. Jika queue kosong dan tujuan belum ditemukan, maka puzzle dianggap tidak memiliki solusi.

Pada BFS, simpul dipilih berdasarkan urutan kedalaman pencarian bukan berdasarkan nilai biaya atau heuristik. Jika dinyatakan dalam bentuk fungsi, BFS dapat dianggap menggunakan:

$g(n)$ = jumlah aksi atau jumlah gerakan dari state awal ke state n

$f(n) = g(n)$

$g(n)$ dalam konteks pemilihan BFS bukanlah total biaya petak, melainkan kedalaman atau jumlah gerakan. Program memang menghitung total cost lintasan untuk ditampilkan sebagai hasil akhir, tetapi nilai tersebut tidak memengaruhi urutan eksplorasi BFS.

BFS optimal jika setiap aksi memiliki sama, biaya yangarena BFS menjamin solusi pertama yang ditemukan adalah solusi dengan jumlah langkah paling sedikit. Akan tetapi, pada permasalahan ini setiap petak dapat memiliki cost yang berbeda. Akibatnya, solusi BFS belum tentu memiliki total biaya (cost) paling kecil. BFS hanya menjamin jumlah gerakan minimum, bukan biaya lintasan minimum.

Dibandingkan dengan UCS, BFS lebih sederhana karena hanya menggunakan queue dan tidak mempertimbangkan cost. Namun, UCS lebih sesuai untuk mencari solusi dengan total biaya minimum karena selalu memilih state dengan cost terkecil. Dibandingkan dengan Greedy Best First Search, BFS tidak menggunakan heuristik sehingga eksplorasinya cenderung lebih luas, tetapi tidak bergantung pada kualitas heuristik. Dibandingkan dengan A*, BFS juga lebih sederhana, tetapi A* dapat lebih efisien apabila heuristik yang digunakan mampu memperkirakan jarak ke tujuan dengan baik.

```

from collections import deque
import time
from board import is_goal
from movement import get_valid_neighbors
from utils import reconstruct_path, make_result, SearchResult

def breadth_first_search(board_info, start_state) -> SearchResult:
    start_time = time.perf_counter()

    queue = deque([start_state])
    visited = {start_state}
    parent = {start_state: None}
    path_cost = {start_state: 0}
    iterations = 0

    while queue:
        current_state = queue.popleft()
        iterations += 1

        if is_goal(board_info, current_state):
            moves, states = reconstruct_path(parent, current_state)
            time_ms = (time.perf_counter() - start_time) * 1000
            return make_result(True,
                               moves,
                               states,
                               path_cost[current_state],
                               iterations,
                               time_ms,)

        for neighbor in get_valid_neighbors(board_info, current_state):
            next_state = neighbor.state
            if next_state in visited:
                continue

            visited.add(next_state)
            parent[next_state] = (current_state, neighbor.direction)
            path_cost[next_state] = path_cost[current_state] + neighbor.move_cost
            queue.append(next_state)

    time_ms = (time.perf_counter() - start_time) * 1000
    return make_result(False, "", [], None, iterations, time_ms)

```

3.2.5. Perbandingan Teoritis Algoritma

Secara teoritis, setiap algoritma pathfinding yang diimplementasikan memiliki strategi pencarian yang berbeda. Perbedaan tersebut memengaruhi optimalitas solusi, jumlah node yang dikunjungi, serta kesesuaian algoritma terhadap permasalahan Ice Sliding Puzzle yang memiliki bobot biaya dan checkpoint.

Algoritma	Dasar Pemilihan Node	Fungsi Evaluasi	Mempertimbangkan Cost	Menggunakan Heuristik	Optimalitas	Karakteristik pada Ice Sliding Puzzle
BFS	Urutan kedalaman atau jumlah langkah paling sedikit	$f(n) = g(n)$, dengan $g(n)$ berupa jumlah langkah	Tidak	Tidak	Optimal jika semua aksi memiliki cost sama	Cocok untuk mencari solusi dengan jumlah langkah minimum, tetapi tidak menjamin cost minimum karena setiap petak dapat

Algoritma	Dasar Pemilihan Node	Fungsi Evaluasi	Mempertimbangkan Cost	Menggunakan Heuristik	Optimalitas	Karakteristik pada Ice Sliding Puzzle
						memiliki biaya berbeda.
UCS	Total cost terkecil dari state awal	$f(n) = g(n)$, dengan $g(n)$ berupa total cost lintasan	Ya	Tidak	Optimal untuk cost non-negatif	Cocok untuk puzzle berbobot karena selalu memilih jalur dengan total cost terkecil, tetapi dapat mengeksplorasi lebih banyak node.
GBFS	Estimasi jarak atau biaya terdekat ke target	$f(n) = h(n)$	Tidak secara langsung	Ya	Tidak menjamin optimal	Dapat menemukan solusi dengan cepat jika heuristik baik, tetapi dapat memilih jalur dengan cost besar karena hanya fokus pada estimasi menuju target.
A*	Gabungan cost yang sudah ditempuh dan estimasi ke target	$f(n) = g(n) + h(n)$	Ya	Ya	Optimal jika heuristik admissible dan cost non-negatif	Cocok untuk puzzle berbobot karena menyeimbangkan cost lintasan dan arah pencarian menuju target.

3.3. Heuristik dan Analisis Admissibility

Pada program ini, heuristik digunakan pada algoritma Greedy Best First Search dan A*. Heuristik berfungsi sebagai nilai perkiraan untuk menentukan seberapa dekat suatu state terhadap target yang sedang dicari. Target pada permainan ini tidak selalu berupa goal akhir, karena pemain harus melewati checkpoint angka secara berurutan terlebih dahulu. Oleh karena itu, apabila masih terdapat checkpoint yang belum dilewati, maka target heuristik adalah checkpoint berikutnya. Setelah seluruh checkpoint selesai dilewati, target heuristik berubah menjadi petak tujuan O.

Dalam implementasi, setiap heuristic menerima state pemain saat ini, posisi goal, data checkpoint, dan informasi papan. Nilai heuristic kemudian digunakan sebagai prioritas pada GBFS atau sebagai bagian dari fungsi evaluasi pada A*. Pada GBFS, nilai heuristic digunakan langsung sebagai prioritas pencarian, sedangkan pada A* nilai heuristic dijumlahkan dengan cost aktual yang sudah ditempuh.

Heuristik yang digunakan pada program ini terdiri dari beberapa pilihan sebagai berikut.

3.3.1. Heuristik Manhattan Distance

Heuristik Manhattan Distance menghitung estimasi jarak berdasarkan selisih baris dan kolom antara posisi pemain dengan target. Heuristik ini cocok digunakan pada papan berbentuk grid karena posisi pada papan dapat dinyatakan sebagai koordinat baris dan kolom.

Secara umum, rumus Manhattan Distance adalah sebagai berikut.

$$h(n) = |\text{row}_n - \text{row_target}| + |\text{col}_n - \text{col_target}|$$

Pada program ini, nilai tersebut dapat dikalikan dengan minimum traversal cost agar estimasi jarak memiliki satuan yang lebih dekat dengan cost permainan. Heuristik ini tidak memperhitungkan rintangan, lava, atau mekanisme sliding secara langsung. Dengan demikian, nilainya hanya menjadi estimasi sederhana dari jarak posisi pemain menuju target.

3.3.2. Heuristik Pythagoras Distance

Heuristik Pythagoras Distance digunakan untuk memperkirakan jarak garis lurus dari posisi pemain menuju target. Heuristik ini mirip dengan Euclidean Distance, tetapi pada laporan ini disebut sebagai Pythagoras Distance agar lebih mudah dipahami karena menggunakan konsep teorema Pythagoras.

Rumus heuristic ini adalah sebagai berikut.

$$h(n) = \text{sqrt}((\text{row}_n - \text{row_target})^2 + (\text{col}_n - \text{col_target})^2)$$

Heuristik ini memberikan estimasi jarak langsung antara dua posisi. Sama seperti Manhattan Distance, heuristic ini tidak memeriksa apakah terdapat dinding, lava, atau aturan sliding di antara posisi pemain dan target. Oleh karena itu, heuristic ini hanya digunakan sebagai perkiraan arah pencarian, bukan sebagai perhitungan cost sebenarnya.

3.3.3. Heuristik Manhattan Pertimbangan Checkpoint

Ini adalah upgrade dari heuristik Manhattan, ide dari heuristic ini adalah state yang masih memiliki banyak checkpoint tersisa dianggap masih lebih jauh dari solusi dibandingkan state yang checkpoint-nya hampir selesai.

Dengan pendekatan ini, nilai heuristic tidak hanya dipengaruhi oleh jarak posisi pemain ke target berikutnya, tetapi juga oleh progress checkpoint. Heuristik ini dapat membantu pencarian agar lebih memperhatikan urutan checkpoint, terutama pada papan yang memiliki lebih dari satu checkpoint.

3.3.4. Heuristik Rantai Checkpoint

Heuristik rantai checkpoint menghitung estimasi jarak dengan mempertimbangkan urutan target yang harus dilewati, mulai dari checkpoint berikutnya hingga goal. Berbeda dengan heuristic yang hanya melihat target terdekat saat ini, heuristic ini mencoba memperkirakan sisa perjalanan secara lebih menyeluruh.

Sebagai contoh, jika pemain masih harus melewati checkpoint 1, checkpoint 2, lalu goal, maka heuristic dapat memperkirakan jarak dari posisi pemain ke checkpoint 1, kemudian dari checkpoint 1 ke checkpoint 2, dan terakhir dari checkpoint 2 ke goal. Dengan demikian, heuristic ini memberikan gambaran yang lebih lengkap mengenai sisa target yang harus dicapai.

Heuristik ini dapat memberi arah pencarian yang lebih baik pada beberapa kasus, terutama ketika checkpoint cukup banyak. Namun, karena perhitungannya masih berbasis estimasi jarak dan tidak sepenuhnya mengikuti mekanisme sliding, nilai heuristic ini tetap tidak selalu sama dengan cost sebenarnya.

3.3.5. Heuristik Sliding Distance

Heuristik Sliding Distance mencoba memperkirakan jarak berdasarkan jumlah gerakan sliding minimum menuju target. Berbeda dengan Manhattan atau Pythagoras yang hanya melihat jarak koordinat, heuristic ini lebih dekat dengan aturan permainan karena mempertimbangkan bahwa pemain bergerak dengan cara sliding, bukan berjalan satu petak.

Pada heuristic ini, program memperkirakan berapa banyak gerakan sliding yang dibutuhkan untuk mencapai target berikutnya. Dengan

pendekatan ini, heuristic dapat lebih sesuai dengan karakteristik Ice Sliding Puzzle dibandingkan jarak koordinat biasa. Namun, heuristic ini juga dapat membutuhkan proses perhitungan tambahan karena harus memperkirakan kemungkinan gerakan pada papan.

Heuristik ini digunakan sebagai salah satu alternatif untuk membandingkan performa pencarian. Pada beberapa kasus, heuristic ini dapat membantu pencarian menjadi lebih terarah, tetapi tetap perlu diuji pada berbagai testcase untuk melihat pengaruhnya terhadap jumlah iterasi dan waktu eksekusi.

3.3.6. Analisis Admissibility

Admissibility adalah sifat heuristik pada A^* yang menunjukkan bahwa nilai $h(n)$ tidak melebihi-lebihkan biaya sebenarnya dari state n menuju target. Jadi, heuristik bisa disebut admissible kalau estimasinya selalu lebih kecil atau sama dengan cost optimal yang sebenarnya.

Pada program ini, Manhattan Distance dan Pythagoras Distance digunakan sebagai estimasi jarak sederhana pada papan. Keduanya hanya melihat jarak posisi pemain ke target, tanpa memperhitungkan rintangan, lava, atau aturan sliding. Karena itu, nilai yang dihasilkan cenderung menjadi estimasi optimistis, terutama jika dikalikan dengan minimum traversal cost.

Namun, Ice Sliding Puzzle tidak sama persis dengan pathfinding grid biasa. Pemain tidak bisa berhenti bebas di setiap petak, tetapi harus terus meluncur sampai tertahan dinding. Akibatnya, posisi yang terlihat dekat secara jarak belum tentu mudah dicapai. Karena itu, Manhattan dan Pythagoras digunakan sebagai heuristik sederhana untuk mengarahkan pencarian, tetapi tetap perlu diuji melalui hasil percobaan.

Heuristik lain yang mempertimbangkan checkpoint atau rantai checkpoint dapat membuat pencarian lebih terarah, tetapi tidak selalu dapat dijamin admissible pada semua kasus. Tambahan estimasi tersebut bisa saja melebihi cost optimal sebenarnya. Oleh karena itu, heuristik tersebut lebih tepat digunakan sebagai pembanding performa.

Secara umum, A^* dapat menghasilkan solusi optimal jika heuristik yang digunakan admissible. Jika heuristik tidak admissible, A^* masih

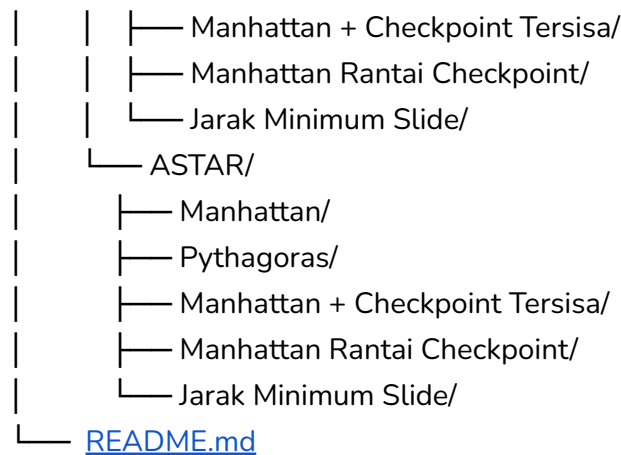
bisa menemukan solusi, tetapi tidak selalu menjamin cost paling kecil. Karena itu, hasil A* dapat dibandingkan dengan UCS, karena UCS dapat digunakan sebagai acuan solusi optimal berdasarkan total cost.

3.4. Struktur Program

Program dibagi ke dalam beberapa file agar setiap bagian memiliki tanggung jawab masing-masing dan lebih mudah dikelola. Struktur program yang digunakan adalah sebagai berikut.

Tucil3_13524015_13524066/

```
|— bin/
|— doc/
|— src/
|   |— main.py
|   |— parser.py
|   |— validator.py
|   |— board.py
|   |— movement.py
|   |— heuristic.py
|   |— utils.py
|   |— algorithms/
|       |— bfs.py
|       |— ucs.py
|       |— gbfs.py
|       |— astar.py
|       |— __init__.py
|— test/
|   |— input/
|       |— input1.txt
|       |— input2.txt
|       |— input3.txt
|       |— input4.txt
|       |— input5.txt
|       |— input6.txt
|   |— output/
|       |— BFS/
|       |— UCS/
|       |— GBFS/
|       |   |— Manhattan/
|       |   |— Pythagoras/
```



- **main.py** berfungsi sebagai file utama yang mengatur alur program, mulai dari membaca input pengguna, memilih algoritma, menjalankan pencarian, hingga menampilkan hasil.
- **parser.py** digunakan untuk membaca file testcase dan mengambil data penting seperti ukuran papan, isi papan, matriks cost, posisi awal, posisi goal, dan checkpoint.
- **validator.py** berfungsi untuk memeriksa apakah input yang dibaca sudah sesuai dengan format dan aturan yang ditentukan.
- **board.py** berisi representasi board dan state permainan, termasuk fungsi untuk mengecek apakah suatu state sudah mencapai goal.
- **movement.py** menangani aturan pergerakan sliding, termasuk menentukan posisi akhir, menghitung cost gerakan, dan mengecek validitas gerakan.
- **heuristic.py** berisi fungsi-fungsi heuristik yang digunakan pada algoritma GBFS dan A*.
- **utils.py** berisi fungsi bantu seperti membentuk ulang solusi, menampilkan hasil, menjalankan playback, mengambil path input, dan menyimpan output.
- **Folder algorithms** berisi implementasi algoritma pathfinding. **ucs.py** berisi implementasi UCS, **gbfs.py** berisi implementasi GBFS, **astar.py** berisi implementasi A*, dan **bfs.py** berisi implementasi BFS sebagai algoritma tambahan.
- **Folder test** berisi tentang test case untuk input dan output.

3.5. Alur Kerja Program

Program dijalankan melalui file **main.py**. Secara umum, alur kerja program dimulai dari pembacaan file input, validasi data, pemilihan algoritma, proses pencarian solusi, hingga penampilan dan penyimpanan hasil.

Secara ringkas, alur kerja program adalah sebagai berikut.

1. Pengguna memasukkan nama file testcase.

2. Program membaca file input.
3. Program melakukan validasi input.
4. Program membentuk board dan state awal.
5. Pengguna memilih algoritma pathfinding.
6. Jika diperlukan, pengguna memilih heuristik.
7. Program menjalankan pencarian solusi.
8. Program membentuk ulang solusi jika goal ditemukan.
9. Program menampilkan solusi, cost, iterasi, waktu, dan visualisasi step.
10. Pengguna dapat menjalankan playback atau menyimpan hasil ke file output.

4. Hasil Pengujian dan Analisis Hasil

4.1. Daftar Test Case

Test Case ke-	Isi Test Case
1: Menguji kasus dasar tanpa checkpoint untuk memastikan setiap algoritma dapat menemukan jalur sederhana dari posisi awal ke tujuan.	5 5 XXXXX XZ**X X***X X**OX XXXXX 999 999 999 999 999 999 1 1 1 999 999 1 1 1 999 999 1 1 1 999 999 999 999 999 999
2: Menguji kemampuan algoritma mengikuti checkpoint secara berurutan sebelum mencapai tujuan akhir.	7 7 XXXXXXX XZ*0*OX X*****X X*****X X*****X X***1X XXXXXXX 999 999 999 999 999 999 999

	999 1 2 3 4 5 999 999 2 3 4 5 6 999 999 3 4 5 6 7 999 999 4 5 6 7 8 999 999 5 6 7 8 9 999 999 999 999 999 999 999 999
3: Menguji validasi gerakan terhadap lava, sehingga algoritma harus menemukan jalur yang tidak melewati petak L.	7 8 XXXXXXXXX XZ*****X X*LXX**X X***L**X X**XX**X X*****OX XXXXXXXXX 999 999 999 999 999 999 999 999 999 1 2 3 4 5 6 999 999 2 999 999 999 7 2 999 999 3 4 5 999 6 3 999 999 4 5 999 999 5 4 999 999 5 4 3 2 1 2 999 999 999 999 999 999 999 999 999
4: Menguji pengaruh bobot biaya pada lintasan, terutama perbedaan hasil antara algoritma yang mempertimbangkan cost dan yang tidak.	5 7 XXXXXXXXX XZ***OX X*****X X*****X XXXXXXXXX 999 999 999 999 999 999 999 999 999 1 9 9 9 1 999 999 1 1 1 1 1 999 999 1 1 1 1 1 999 999 999 999 999 999 999 999 999
5: Menguji kondisi ketika puzzle tidak memiliki solusi, sehingga program harus mengembalikan hasil bahwa solusi tidak ditemukan.	6 7 XXXXXXXXX XZ***OX XXX*XXX X0***1X XXX*XXX XXXXXXXXX 999 999 999 999 999 999 999 999 999 1 1 1 1 1 999 999 999 999 2 999 999 999 999 3 3 3 3 3 999

	999 999 999 2 999 999 999 999 999 999 999 999 999 999
6: Menguji kasus yang lebih kompleks dengan ukuran papan lebih besar dan beberapa checkpoint berurutan.	9 9 XXXXXXXXXX XZ**0**OX X*****X X*****X X*****X X*****X X*****X X*****X X*2****1X XXXXXXXXXX 999 999 999 999 999 999 999 999 999 999 1 2 3 4 3 2 1 999 999 1 3 4 5 4 3 2 999 999 1 4 5 6 5 4 3 999 999 1 5 6 7 6 5 4 999 999 1 4 5 6 5 4 3 999 999 1 3 4 5 4 3 2 999 999 1 2 3 4 3 2 1 999 999 999 999 999 999 999 999 999 999

4.2. Hasil Output

Hasil screenshot output akan dilampirkan untuk setiap test-case tetapi setiap test case tidak akan dilampirkan keseluruhan dari outputnya. Jadi, hanya sekitar 1 hingga 2 agar tidak terlalu banyak screenshot dari output. Hasil statistik output lengkapnya akan dilampirkan pada segmen selanjutnya. Oleh karena itu, dipastikan setiap dari pilihan algoritma akan terdapat tesnya minimal satu kali

Test Case	Pilihan Input	Hasil
-----------	---------------	-------

1	UCS	<pre>Solusi Yang Ditemukan : DR Cost dari Solusi : 4 Waktu eksekusi: 0.197 ms Banyak iterasi: 4 iterasi Initial XXXXX XZ**X X***X X**OX XXXXX Step 1 : D XXXXX X***X X***X XZ*OX XXXXX Step 2 : R XXXXX X***X X***X X**ZX XXXXX</pre>
---	-----	--

2	GBFS, Manhattan	Solusi Yang Ditemukan : RDU Cost dari Solusi : 70 Waktu eksekusi: 0.611 ms Banyak iterasi: 4 iterasi Initial XXXXXXXX XZ*0*0X X*****X X*****X X*****X X*****1X XXXXXXXX Step 1 : R XXXXXXXX X**0*ZX X*****X X*****X X*****X X*****1X XXXXXXXX Step 2 : D XXXXXXXX X**0*0X X*****X X*****X X*****X X*****ZX XXXXXXXX Step 3 : U XXXXXXXX X**0*ZX X*****X X*****X X*****X X*****1X XXXXXXXX
---	--------------------	---

3	A*, Pythagoras	<pre> Solusi Yang Ditemukan : DR Cost dari Solusi : 26 Waktu eksekusi: 0.502 ms Banyak iterasi: 4 iterasi Initial XXXXXXXX XZ*****X X*LXX**X X***L**X X**XX**X X*****OX XXXXXXXX Step 1 : D XXXXXXXX X*****X X*LXX**X X***L**X X**XX**X XZ*****OX XXXXXXXX Step 2 : R XXXXXXXX X*****X X*LXX**X X***L**X X**XX**X X*****ZX XXXXXXXX </pre>
4	BFS	<pre> Solusi Yang Ditemukan : R Cost dari Solusi : 28 Waktu eksekusi: 0.301 ms Banyak iterasi: 3 iterasi Initial XXXXXX XZ***OX X*****X X*****X XXXXXX Step 1 : R XXXXXX X*****ZX X*****X X*****X XXXXXX </pre>

5	A*, Manhattan rantai checkpoint	Solusi tidak ditemukan. Waktu eksekusi: 0.291 ms Banyak iterasi: 2 iterasi
6	GBFS, Manhattan + checkpoint tersisa	Solusi Yang Ditemukan : RDULDRU Cost dari Solusi : 96 Waktu eksekusi: 1.493 ms Banyak iterasi: 9 iterasi <pre> Initial XXXXXXXXXX XZ**0**0X X*****X X*****X X*****X X*****X X*****X X*****X X*2***1X XXXXXXXXXX Step 1 : R XXXXXXXXXX X***0**ZX X*****X X*****X X*****X X*****X X*****X X*2***1X XXXXXXXXXX Step 2 : D XXXXXXXXXX X***0**0X X*****X X*****X X*****X X*****X X*****X X*2***ZX XXXXXXXXXX Step 3 : U XXXXXXXXXX X***0**ZX X*****X X*****X X*****X X*****X X*****X X*2***1X XXXXXXXXXX </pre>

		Step 4 : L XXXXXXXXX XZ**0**0X X*****X X*****X X*****X X*****X X*****X X*2****1X XXXXXXXXX Step 5 : D XXXXXXXXX X***0**0X X*****X X*****X X*****X X*****X X*****X XZ2****1X XXXXXXXXX Step 6 : R XXXXXXXXX X***0**0X X*****X X*****X X*****X X*****X X*****X X*2****ZX XXXXXXXXX Step 7 : U XXXXXXXXX X***0**ZX X*****X X*****X X*****X X*****X X*2****1X XXXXXXXXX
--	--	---

3	A*, Jarak minimum slide ke target berikutnya	Solusi Yang Ditemukan : DR Cost dari Solusi : 26 Waktu eksekusi: 24.810 ms Banyak iterasi: 4 iterasi <pre> Initial XXXXXXXX XZ***** X*LXX**X X***L**X X**XX**X X*****OX XXXXXXXX Step 1 : D XXXXXXXX X*****X X*LXX**X X***L**X X**XX**X XZ***OX XXXXXXXX Step 2 : R XXXXXXXX X*****X X*LXX**X X***L**X X**XX**X X*****ZX XXXXXXXX </pre>
---	--	--

4.3. Statistik Pengujian

Pengujian algoritma dan heuristic menggunakan test case 1 - 6. Hasil statistik dari program berdasarkan algoritma pathfinding dan heuristic yang digunakan dilampirkan pada tabel berikut.

Algoritma pathfinding dan heuristic	TC	Cost	Langkah	Node	Waktu
UCS	1	4	2	4	0.113
	2	70	3	7	0.212
	3	26	2	4	0.125
	4	8	3	4	0.105
	5	-	-	2	0.055
	6	66	5	12	0.426
GBFS, Manhattan	1	4	2	3	0.388

	2	70	3	4	0.262
	3	31	2	3	0.184
	4	28	1	2	0.092
	5	-	-	2	0.084
	6	96	7	9	0.683
GBFS, Pythagoras	1	4	2	3	0.113
	2	70	3	4	0.251
	3	31	2	3	0.168
	4	28	1	2	0.092
	5	-	-	2	0.076
	6	96	7	9	0.722
GBFS, Manhattan + Checkpoint	1	4	2	3	0.099
	2	70	3	4	0.495
	3	31	2	3	0.165
	4	28	1	2	0.079
	5	-	-	2	0.072
	6	96	7	9	0.708
GBFS, Manhattan Rantai	1	4	2	3	0.102
	2	70	3	4	0.458
	3	31	2	3	0.161
	4	28	1	2	0.080
	5	-	-	2	0.074
	6	66	5	6	0.539
GBFS, Jarak Minimum Slide	1	4	2	3	0.258
	2	70	3	5	1.751
	3	26	2	3	0.324
	4	28	1	2	0.227
	5	-	-	2	0.156
	6	96	7	9	3.154
A*, Manhattan	1	4	2	4	0.126
	2	70	3	7	0.483
	3	26	2	4	0.177

	4	8	3	4	0.173
	5	-	-	2	0.086
	6	66	5	11	0.756
A*, Pythagoras	1	4	2	4	0.121
	2	70	3	7	0.384
	3	26	2	4	0.203
	4	8	3	4	0.174
	5	-	-	2	0.124
	6	66	5	11	0.789
A*, Manhattan + Checkpoint	1	4	2	4	0.120
	2	70	3	7	0.355
	3	26	2	4	0.196
	4	8	3	4	0.184
	5	-	-	2	0.079
	6	66	5	11	0.804
A*, Manhattan Rantai	1	4	2	4	0.197
	2	70	3	7	0.358
	3	26	2	4	0.199
	4	8	3	4	0.170
	5	-	-	2	0.098
	6	66	5	10	0.715
A*, Jarak Minimum Slide	1	4	2	4	0.589
	2	70	3	7	1.273
	3	26	2	4	0.415
	4	8	3	4	0.355
	5	-	-	2	0.190
	6	66	5	12	3.432
BFS	1	4	2	4	0.411
	2	70	3	6	0.196
	3	26	2	4	0.139
	4	28	1	3	0.098
	5	-	-	2	0.079
	6	66	5	13	0.588

4.4. Analisis Performa Hasil

4.4.1. Analisis Algoritma

Breadth-First Search (BFS) berhasil menemukan solusi pada TC 1, TC 2, TC 3, TC 4, dan TC 6, tetapi tidak menemukan solusi pada TC 5 karena test case tersebut memang tidak memiliki jalur valid. BFS cenderung menghasilkan solusi dengan jumlah langkah paling sedikit, seperti pada TC 4 yang hanya membutuhkan 1 langkah. Namun, solusi tersebut memiliki cost 28, lebih besar dibanding UCS dan A* yang menghasilkan cost 8. Hal ini menunjukkan bahwa BFS tidak cocok untuk mencari solusi termurah pada kasus berbobot karena tidak mempertimbangkan cost dalam pemilihan node.

Uniform Cost Search (UCS) menghasilkan solusi dengan cost minimum pada test case berbobot. Hal ini terlihat jelas pada TC 4, ketika UCS memilih solusi dengan cost 8 meskipun membutuhkan 3 langkah, sedangkan BFS memilih solusi 1 langkah dengan cost 28. UCS memilih node berdasarkan total biaya dari awal, sehingga lebih sesuai untuk permasalahan yang memiliki bobot biaya pada setiap petak. Kekurangannya, UCS dapat mengunjungi lebih banyak node dibanding algoritma berbasis heuristik karena tidak memiliki informasi estimasi menuju tujuan.

Greedy Best First Search (GBFS) cenderung mengunjungi node lebih sedikit karena memilih node berdasarkan nilai heuristik saja. Pada TC 1 sampai TC 4, GBFS umumnya hanya mengunjungi 2 sampai 4 node. Namun, kualitas solusi GBFS sangat bergantung pada heuristik yang digunakan. Pada TC 6, GBFS H1, H2, H3, dan H5 menghasilkan cost 96, sedangkan GBFS H4 menghasilkan cost 66. Hal ini menunjukkan bahwa GBFS dapat cepat, tetapi tidak menjamin solusi optimal.

A* menghasilkan solusi yang lebih stabil dibanding GBFS karena mempertimbangkan dua komponen, yaitu cost yang sudah ditempuh dan estimasi menuju tujuan. Pada TC 4, seluruh variasi A* menghasilkan cost 8, sama seperti UCS. Pada TC 6, seluruh variasi A* juga menghasilkan cost 66. Perbedaan utama antarheuristik pada A* terlihat pada jumlah node yang dikunjungi dan waktu eksekusi. Secara umum, A* lebih cocok dibanding GBFS untuk mencari solusi optimal pada permasalahan berbobot.

4.4.2. Analisis Heuristik

Heuristik H1 Manhattan memberikan estimasi berdasarkan jarak grid horizontal dan vertikal menuju target berikutnya. Pada A*, H1 menghasilkan solusi optimal pada semua test case yang memiliki solusi. Namun, pada GBFS, H1 tidak selalu menghasilkan cost terbaik, misalnya pada TC 6 menghasilkan cost 96. Hal ini menunjukkan bahwa Manhattan cukup baik sebagai estimasi dasar, tetapi kurang memperhatikan struktur checkpoint secara menyeluruh.

Heuristik H2 Pythagoras menggunakan jarak garis lurus menuju target berikutnya. Hasil H2 mirip dengan H1 pada sebagian besar test case. Pada A*, H2 tetap menghasilkan solusi optimal, sedangkan pada GBFS H2 menghasilkan cost yang sama dengan H1 pada TC 6, yaitu 96. Hal ini menunjukkan bahwa meskipun Pythagoras dapat memperkirakan kedekatan geometris, heuristik ini belum tentu sesuai dengan aturan sliding dan checkpoint pada puzzle.

Heuristik H3 Manhattan + Checkpoint Tersisa menambahkan jumlah checkpoint yang belum dilewati ke dalam estimasi Manhattan. Pada hasil pengujian, H3 tidak memberikan peningkatan signifikan dibanding H1 dan H2. Pada TC 6, GBFS H3 tetap menghasilkan cost 96, sama seperti H1 dan H2. Hal ini menunjukkan bahwa menambahkan jumlah checkpoint saja belum cukup untuk mengarahkan pencarian ke jalur terbaik jika urutan dan posisi checkpoint tidak diperhitungkan secara lebih detail.

Heuristik H4 Manhattan Rantai Checkpoint memberikan hasil terbaik pada kasus yang melibatkan beberapa checkpoint. Pada TC 6, GBFS H4 menghasilkan cost 66 dengan 5 langkah dan hanya 6 node dikunjungi, lebih baik dibanding GBFS H1, H2, H3, dan H5 yang menghasilkan cost 96. Pada A*, H4 juga menghasilkan cost optimal dengan jumlah node paling sedikit pada TC 6, yaitu 10 node. Hal ini menunjukkan bahwa H4 paling sesuai untuk puzzle dengan checkpoint karena memperkirakan jalur dari posisi sekarang ke checkpoint berikutnya hingga tujuan akhir.

Heuristik H5 Jarak Minimum Slide memperkirakan jarak berdasarkan jumlah gerakan sliding minimum menuju target berikutnya. Pada TC 3, GBFS H5 menghasilkan cost 26, lebih baik dibanding GBFS H1 sampai H4 yang menghasilkan cost 31. Namun, H5 memiliki waktu eksekusi lebih tinggi, terutama pada TC 6, karena perhitungan

heuristiknya membutuhkan pencarian tambahan. Dengan demikian, H5 dapat memberi estimasi yang lebih sesuai terhadap aturan sliding, tetapi memiliki biaya komputasi lebih besar.

5. Kesimpulan dan Saran

5.1. Kesimpulan

Dalam proyek Tugas Kecil 3 IF2211 Strategi Algoritma ini, algoritma pathfinding berupa BFS, UCS, GBFS, dan A* dapat digunakan dalam penyelesaian Ice Sliding Puzzle. Setiap algoritma mampu melakukan pencarian jalur dari posisi awal menuju tujuan dengan mengikuti aturan permainan, seperti mekanisme sliding, penghalang, lava, checkpoint, dan cost pada setiap petak.

Kami berhasil mengimplementasikan algoritma pathfinding dengan pendekatan pencarian pada ruang state. Setiap state merepresentasikan posisi pemain dan checkpoint berikutnya yang harus dilewati. Program juga berhasil membangkitkan gerakan valid, menghitung cost lintasan, menyimpan parent untuk rekonstruksi solusi, serta menampilkan hasil berupa urutan gerakan, total cost, jumlah node yang dikunjungi, dan waktu eksekusi.

Berdasarkan hasil pengujian, UCS dan A* lebih sesuai untuk mencari solusi dengan cost minimum karena mempertimbangkan biaya lintasan. BFS dapat menemukan solusi dengan jumlah langkah minimum, tetapi tidak selalu menghasilkan cost terkecil. GBFS dapat mengunjungi node lebih sedikit, tetapi hasilnya sangat bergantung pada heuristik yang digunakan. Pada pengujian dengan checkpoint, heuristik Manhattan Rantai Checkpoint memberikan hasil yang paling baik karena memperhitungkan urutan checkpoint sampai tujuan akhir.

5.2. Saran

Untuk pengembangan lebih lanjut, program dapat dilengkapi dengan antarmuka grafis agar proses pencarian dan pergerakan pemain dapat divisualisasikan dengan lebih jelas. Dengan adanya visualisasi, pengguna dapat melihat perubahan posisi pemain pada setiap langkah solusi secara lebih mudah.

Selain itu, heuristik yang digunakan masih dapat dikembangkan agar lebih sesuai dengan karakteristik Ice Sliding Puzzle. Heuristik dapat dibuat dengan mempertimbangkan dinding, lava, arah sliding, serta urutan checkpoint secara lebih akurat, sehingga pencarian dapat menjadi lebih efisien tanpa mengorbankan kualitas solusi.

Pengujian juga dapat diperluas dengan jumlah test case yang lebih banyak dan bervariasi, seperti papan berukuran lebih besar, jumlah checkpoint lebih banyak, serta konfigurasi cost yang lebih kompleks. Dengan demikian, perbandingan performa antara algoritma dan heuristik dapat dianalisis dengan lebih menyeluruh.

6. Daftar Pustaka

- [1] D. Tamara, "BFS (Breadth First Search): Pengertian, Kekurangan, Kelebihan, dan Contohnya," Medium, Oct. 15, 2021. [Online]. Available: <https://medium.com/@defytamara2610/bfs-breadth-first-search-pengertian-kekurangan-kelebihan-dan-contohnya-775a7d808fbc> . [Accessed: May 6, 2026].
- [2] R. Munir, "BFS/DFS (2026) Bagian 1," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/13-BFS-DFS-(2026)-Bagian1.pdf) [Accessed: May 10, 2026].
- [3] R. Munir, "BFS/DFS (2026) Bagian 2," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/14-BFS-DFS-(2026)-Bagian2.pdf) . [Accessed: May 6, 2026].
- [4] N. U. Maulidevi, "Penentuan Rute (Route/Path Planning), Bagian 1: BFS, DFS, UCS, Greedy Best First Search," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-(2026)-Bagian1.pdf) . [Accessed: May 6, 2026].
- [5] N. U. Maulidevi and R. Munir, "Penentuan Rute (Route/Path Planning), Bagian 2: Algoritma A*," Program Studi Teknik Informatika, Institut Teknologi Bandung, 2026.

[Online]. Available:
[https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-(2026)-Bagian2.pdf) . [Accessed: May 6, 2026].

[6] R. Dechter, "Lecture Notes in Informed Heuristic Search," ICS 271, University of California, Irvine, 2008. [Online]. Available:
<http://www.ics.uci.edu/~dechster/courses/ics-271/fall-08/lecture-notes/4.InformedHeuristicSearch.ppt> . [Accessed: May 6, 2026].

7. Lampiran

7.1. Pranala Repository Github

https://github.com/NathanaelGun/Tucil3_13524015_13524066

7.2. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Mahatma Brahmana



Nathanael Gunawan

7.3. Checklist

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	

3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)		✓
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)	✓ (Hanya satu karena kelalaian membaca spek hanya pada bagian bonus yang tidak tertulis dua)	
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	